The Stata Journal (2026) **00**, Number 00

wbopendata: Programmatic Access to World Bank Open Data

João Pedro Azevedo
UNICEF
Division of Data, Analytics, Planning and Monitoring
3 United Nations Plaza
New York, NY 10017
jpazevedo@unicef.org

Abstract. This article examines data provenance in the age of automation, drawing on fifteen years of development and sustained use of `wbopendata`, the first Stata command to provide programmatic access to an international development data repository. Today, `wbopendata` provides access to over 29,000 indicators from 71 databases spanning 296 countries and aggregates, with features including five download modes, multilingual metadata, publication-ready graph formatting, and—as of version 18.0—offline indicator discovery through a YAML-backed metadata architecture and version-controlled metadata synchronization. Its broader contribution lies in treating data acquisition as code: indicator selections, country filters, and time ranges become explicit parameters in analysis scripts rather than undocumented manual downloads—addressing the data provenance dimension of the reproducibility crisis that statistical reforms alone cannot fix. Yet the command has never been more relevant. As AI tools accelerate analytical workflows while enabling plausible fabrication of statistics and citations, anchoring research to authoritative, version-controlled data sources becomes essential infrastructure—not legacy convenience. I document the command's latest syntax and stored results, demonstrate workflows from basic queries to choropleth mapping, and present a 163-scenario, two-layer test suite. I also discuss risks that frictionless data access can obscure—provenance opacity, coverage gaps masked by convenient defaults, and sustainability pressures—alongside mitigation strategies. I distill three design principles from fifteen years of sustained use: backward compatibility builds trust, domain-specific syntax lowers barriers, and scripted data access makes reproducibility the default rather than the exception.

Keywords: st0001, wbopendata, Open Data, World Bank, APIs, reproducible research, development indicators

1 Introduction

In April 2010, the World Bank launched its Open Data Initiative (**?**), transforming decades of development statistics into a freely accessible global public good. By coupling an open data portal with a programmatic Application Programming Interface (API), the initiative reframed official statistics from downloadable artifacts into queryable services. This shift altered not only how data are disseminated, but how empirical research can

be conducted.

Within less than a year, `wbopendata` (**?**) was released as a Stata command translating this new API into a domain-specific interface for applied researchers. Over the subsequent fifteen years, both the World Bank's data infrastructure and `wbopendata` evolved substantially: the number of databases expanded, legacy endpoints were retired, metadata standards matured, and new requirements emerged around multilingual documentation, versioning, and reproducibility. Throughout these changes, `wbopendata` maintained backward compatibility while embedding increasingly sophisticated mechanisms for scripted data access, metadata management, and automated documentation.

This article advances three claims. First, a substantial share of the reproducibility crisis in the social sciences stems not from statistical methodology, but from opaque data acquisition practices. Manual downloads, undocumented filters, and ad hoc preprocessing sever the link between published results and their underlying data sources. Second, treating data acquisition as code—where indicator selection, country coverage, and time ranges are explicit, parameterized, and executable—offers a practical response to this problem. `wbopendata` operationalizes this principle by making data provenance an integral component of the analytical script rather than an external narrative. Third, in an era of rapidly expanding AI-assisted research, such constraints are becoming more, not less, important. As generative tools lower the cost of producing plausible analyses and narratives, anchoring empirical work to authoritative and verifiable data sources becomes essential infrastructure for scientific credibility.

Against this backdrop, `wbopendata` is best understood not as a convenience tool, but as a constraint mechanism. By binding analysis to a single authoritative API and exposing all data selection decisions as executable code, it limits the space of admissible data inputs in ways that complement—but also discipline—AI-assisted workflows. The command ensures that acceleration downstream does not come at the cost of unverifiable or opaque data upstream.

The remainder of the article documents the current capabilities of `wbopendata`, demonstrates reproducible analytical workflows, and reflects on lessons learned from fifteen years of sustained use. Section **??** introduces the design principles and scope of the command. Section **??** documents its syntax and stored results. Section **??** presents canonical workflows for reproducible analysis and visualization. Section **??** describes the technical implementation and test suite. Section **??** discusses broader implications for reproducibility in the context of AI-assisted research, and Section **??** concludes.

2 Design principles and scope

The fifteen-year evolution of `wbopendata` reflects design choices shaped by sustained use in applied research environments rather than abstract software engineering goals. These choices respond to practical constraints common in work with official statistics: heterogeneous upstream producers, revisions to published series, institutional computing restrictions, and the need to document provenance transparently. This section distills

three design principles embedded in the command and clarifies its intended scope.

2.1 Data acquisition as code

The foundational principle of `wbopendata` is that data acquisition should be treated as part of the analytical codebase rather than as an external preparatory step. Indicator selection, country coverage, time ranges, and filters are expressed as explicit command parameters that can be executed, inspected, and version-controlled alongside the analysis itself.

This principle has antecedents in the Stata ecosystem. **?** demonstrated how a purpose-built command (`freduse`) could transform the retrieval of Federal Reserve economic data from a manual browser task into a scriptable, reproducible operation embedded in analytical code. `wbopendata` extends this logic to the broader landscape of World Bank development indicators, applying the same core insight—that data acquisition should be code—to a data ecosystem an order of magnitude larger.

This approach contrasts with common workflows in which researchers manually download spreadsheets from web portals, rename files, apply undocumented filters, and import the results into statistical software. Such workflows often leave no complete record of how the analytical dataset was constructed, making replication difficult even when code and data files are shared. By encoding acquisition decisions directly in Stata syntax, `wbopendata` produces an executable specification of provenance: a command line documents what data were requested, from which source, and under what constraints.

This principle does not guarantee identical data across time. Instead, it guarantees traceability. If upstream data are revised, the same command yields systematically updated results rather than silently diverging datasets. Reproducibility, in this sense, includes both exact replication (when sources are unchanged) and transparent updating (when new observations or revisions are introduced).

2.2 Backward compatibility as trust infrastructure

A second guiding principle is the prioritization of backward compatibility. Over the past decade, the World Bank API has undergone substantial changes, including endpoint deprecations, catalog restructuring, and metadata revisions. Throughout these transitions, `wbopendata` has aimed to preserve stable syntax and predictable behavior wherever possible.

Backward compatibility serves a methodological function beyond user convenience. Researchers build analytical pipelines that may be rerun years after initial publication, often by different teams. When data access commands change semantics or fail without warning, reproducibility is compromised even if the original code is preserved. By absorbing API changes within the command's internal logic—rather than propagating them to the user interface—`wbopendata` reduces the risk that historical analyses become

irreproducible due to infrastructure drift.

This commitment imposes constraints. New features are introduced cautiously, defaults are chosen conservatively, and deprecated behavior is handled through informative diagnostics. The result is a command that evolves incrementally while maintaining continuity, fostering long-term trust among users who rely on it as part of production workflows.

2.3 Domain-specific syntax as error prevention

The third design principle is the use of domain-specific syntax tailored to applied development research. Rather than exposing users directly to HTTP requests, JSON parsing, pagination logic, or API schemas, `wbopendata` presents an interface organized around concepts familiar to its audience: indicators, countries, years, topics, and metadata.

This choice is not merely ergonomic. By constraining user input to semantically meaningful parameters, the command reduces opportunities for error and misinterpretation. Indicator codes must be valid; country identifiers must conform to documented standards; time ranges are explicit. These constraints limit the space of admissible queries and make incorrect or ambiguous data requests easier to detect.

In this sense, domain-specific syntax functions as a guardrail. While generic API clients offer maximal flexibility, they also place the burden of correctness entirely on the user. `wbopendata` trades some generality for a higher likelihood that queries correspond to interpretable, well-documented data products.

2.4 Scope, boundaries, and diffusion

Scope and non-goals

Clarifying scope is essential to understanding the role of `wbopendata` in reproducible research. The command is designed to provide programmatic access to *aggregated* development indicators disseminated through the World Bank API, together with their associated metadata. It is not a tool for accessing confidential or licensed microdata, nor does it perform statistical harmonization, imputation, or methodological adjustments beyond those already embodied in the source databases.

Similarly, `wbopendata` does not aim to abstract away substantive judgment. Choices about indicator suitability, methodological breaks, coverage gaps, and interpretation remain the responsibility of the researcher. The command supports these judgments by surfacing metadata and coverage diagnostics, but it does not substitute for domain expertise.

Within these boundaries, `wbopendata` is best understood as infrastructure rather than analysis software. Its contribution lies in constraining and documenting the earliest stage of the empirical workflow—data acquisition—so that subsequent analysis, whether conducted manually or assisted by automated tools, rests on verifiable and reproducible

foundations.

Diffusion beyond `wbopendata`

The design principles outlined above are not unique to `wbopendata`. They reflect a broader pattern of tooling that has emerged within applied development research to address reproducibility, provenance, and scale. In particular, similar principles have guided the development of other Stata-based data access tools, both within and beyond the World Bank.

Within the World Bank, these ideas informed the evolution of internal data infrastructure such as `datalib`, `datalib2`, and `datalibweb` (**?**), which provide structured, version-controlled access to raw and harmonized household survey microdata. While differing substantially in scope and access restrictions from `wbopendata`, these tools apply the same core logic: data retrieval is scripted, parameterized, and embedded directly in analytical workflows. This approach has enabled large-scale, reproducible analytical systems, most notably the Poverty and Inequality Platform (PIP) (**?**), a publicly accessible global platform that disseminates official poverty and inequality estimates produced through transparent and replicable pipelines.

An early and influential precedent is `freduse` (**?**), which demonstrated that economic data retrieval could be embedded directly in Stata syntax with per-variable metadata stored as variable characteristics (`char`). The design choices visible in `freduse`—scripted acquisition, structured metadata, and Stata-native output—anticipate many of the patterns adopted by `wbopendata` and its successors.

Outside the World Bank, related principles can be observed in user-written Stata tools such as `datazoom` (**?**), which systematizes the preparation of Brazilian household survey microdata produced by the national statistical agency. Although `datazoom` operates on locally obtained files rather than remote APIs, it similarly emphasizes standardized structure, transparent preprocessing, and reusable code as prerequisites for credible empirical analysis.

More recently, the same design logic has been extended to multi-language data access through `unicefData` (**?**), which provides a triangulated suite of R, Python, and Stata interfaces to UNICEF's SDMX-based data warehouse. While implemented in different programming environments, these tools share a common emphasis on explicit data acquisition, metadata exposure, and reproducible defaults. The influence has also been bidirectional: the discovery architecture introduced by `unicefData`—offline catalog browsing with YAML-backed metadata and clickable SMCL navigation—was subsequently adopted by `wbopendata` (v18.0), demonstrating the portability of these design patterns across data ecosystems.

These examples are not intended as an exhaustive survey of data infrastructure tools, nor do they imply architectural equivalence. Rather, they illustrate that the principles embedded in `wbopendata` have proven portable across institutions, data modalities, and governance contexts. Their recurrence suggests that treating data access as constrained,

executable infrastructure—rather than an informal preprocessing step—is a generalizable response to the reproducibility challenges facing empirical research with official statistics.

3 The wbopendata command

The databases accessible through `wbopendata` include World Development Indicators (WDI), Doing Business, Worldwide Governance Indicators, International Debt Statistics, Africa Development Indicators, Education Statistics, Enterprise Surveys, Gender Statistics, Health Nutrition and Population Statistics, Global Financial Inclusion (Findex), Poverty and Equity, Human Capital Index, Sustainable Development Goals, and many more. Table **??** summarizes the current scope.

Table 1: World Bank Open Data coverage

Dimension	Coverage
Indicators	29,000+
Data sources	71 databases
Topic categories	21
Countries & regions	296
Country attributes	17
Time coverage	1960–present
Languages	3 (English, Spanish, French)

`wbopendata` talks directly to the World Bank API (JSON over HTTP), returns tidy Stata datasets, and caches results to minimize repeat downloads. Five pull modes cover country, topic, single-indicator (all countries), single-indicator (selected countries), and multi-indicator requests. Output may be wide or long; `latest` works in long mode and returns ready-to-use scalars for titles and subtitles. Metadata is always fetched; v17.7 adds basic country context (region, admin region, income level, lending type) by default. Version 18.0 introduces discovery commands for offline catalog browsing, a YAML-backed metadata architecture with over 29,000 indicator records, and a sync system for version-controlled metadata updates. Version 18.1 adds variable-level characteristic metadata for self-documenting datasets and deterministic offline testing. Version 18.2 introduces a local data response cache with configurable time-to-live, and version 18.3 adds cache-aware discovery and cache management refinements. Version 18.4 refactors internal ado-file naming to comply with Stata Journal conventions, replacing single-underscore prefixes with the `__wbod_` namespace to avoid conflicts with official Stata routines.

3.1 Syntax

`wbopendata, { indicator(`*string*`) | country(`*string*`) | topics(`*string*`) }`
`[ options ]`

Data selection

Exactly one of the following is required:

`indicator(`*string*`)` specifies one or more World Bank indicator codes. Multiple indicators can be requested by separating codes with semicolons:
`indicator(SP.POP.TOTL;NY.GDP.PCAP.CD)`.

`country(`*string*`)` specifies ISO3 country codes or World Bank region codes to retrieve all available indicators for selected countries. Multiple codes can be separated by semicolons.

`topics(#)` specifies a topic ID (1–21) to retrieve all indicators within a thematic category such as education, health, or environment.

Time and language

`year(`*string*`)` restricts the time interval. For example, `year(2000:2020)` returns data only for years 2000 through 2020.

`date(`*string*`)` provides an alternative time filter for series that support quarterly or monthly periodicity (e.g., `date(2020Q1:2021Q4)`).

`source(`*string*`)` restricts the query to a specific World Bank data source by numeric ID (e.g., `source(2)` for World Development Indicators).

`language(`*string*`)` sets the language for metadata display. Valid codes are `en` (English, default), `es` (Spanish), and `fr` (French).

`projection` accesses population estimates and projections from the Health Nutrition and Population Statistics database rather than actual census data.

Output format

`long` returns data in long format with one row per country-year. The default is wide format with year-specific columns (`yr1960`, `yr1961`, ...).

`clear` replaces any data currently in memory. Required if data are already loaded.

`latest` keeps only the most recent non-missing observation per country. Requires the `long` option. When multiple indicators are requested, retains only observations where *all* indicators have non-missing values in the same year.

`describe` displays indicator metadata without downloading data. Useful for exploring indicator definitions and sources before committing to a full download.

`nometadata` suppresses the metadata display that normally appears after data retrieval.

`nochar` suppresses the dataset and variable characteristics (`char`) that are written by default as of v18.1. See Section **??** for details on the characteristic metadata layer.

Country attributes

`basic` adds region, administrative region, income level, and lending type variables to the downloaded data. This is the default behavior in v17.7+.

`nobasic` suppresses the default country attribute variables.

`full` adds all 17 country attributes including geographic coordinates and capital city. See Table **??** for the complete list.

`iso`, `regions`, `adminr`, `income`, `lending` add individual attribute groups without the full set.

`geo`, `capital`, `latitude`, `longitude` add specific geographic fields without the full set of attributes.

`match(`*varname*`)` merges country attributes into an existing dataset. The variable *varname* must contain World Bank country codes (ISO3 format).

Metadata management (v18.0+)

Sync preview (dry run):

`sync` previews metadata changes without applying them. Compares the local cache version against the latest available release and displays a summary. This is the safe default—no files are modified.

`sync detail` extends the preview with per-source and per-topic indicator breakdowns.

`sync force` forces a fresh preview by re-querying the API rather than relying on cached diagnostics. Still a dry run.

Sync apply:

`sync replace` applies the metadata synchronization—downloads the latest YAML metadata release from the GitHub repository and updates the local cache.

`sync replace force` forces a re-download regardless of the local cache version, bypassing staleness checks.

Cache management:

`checkupdate` queries the remote repository for the latest release version and reports whether an update is available, without downloading or modifying any files.

`cacheinfo` displays the cache directory location, current metadata version, and last synchronization timestamp.

`clearcache` removes the local metadata cache entirely, forcing a full re-download on the next sync or discovery command.

Data response cache (v18.2+):

`nocache` bypasses the local data response cache for a single query, forcing a fresh down-

load from the World Bank API regardless of whether a cached response exists.

`cachedays(#)` sets the time-to-live for cached data responses in days. The default is 7. For example, `cachedays(30)` retains cached API responses for 30 days before re-fetching.

`cleardatacache` removes all cached API response files from the local data cache directory.

`resetdatacache` expires all cached data entries without deleting the underlying files. The next query for any indicator will re-fetch fresh data from the API, but the old cache files remain available for inspection.

`verbose` enables targeted diagnostic output for cache and metadata operations, displaying cache hit/miss status, file paths, and timing information.

Graph metadata (v17.6+)

`linewrap(`*string*`)` wraps metadata text for use in graph titles. The argument specifies which metadata to wrap: `name`, `description`, `note`, `source`, `topic`, or `all`.

`maxlength(#)` sets the maximum characters per line for wrapped text. The default is 50.

`linewrapformat(`*string*`)` controls the output format: `stack` (stacked lines), `newline` (newline-separated), `nlines` (returns line count), `lines` (returns individual lines), or `all` (returns all formats).

3.2 Discovery commands (v18.0)

Version 18.0 introduces a suite of discovery commands that allow users to explore the World Bank Open Data catalog without downloading data. The discovery architecture follows the model introduced by `unicefdata` (`?`), which pioneered offline catalog browsing with YAML-backed metadata and clickable SMCL navigation in Stata. The `wbopendata` implementation extends this pattern to the World Bank's 71 data sources and 29,000+ indicators.

All discovery outputs feature clickable SMCL links that let users drill down from sources to topics to individual indicators, and then download data with a single click.

`sources` lists all World Bank data sources (databases) with indicator counts and clickable navigation links.

`allsources` lists all data sources without the default display limit, showing the complete set of 71 databases.

`alltopics` lists all 21 World Bank topic categories with indicator counts and clickable navigation links.

`search(`*string*`)` searches indicators by keyword, wildcard pattern, or regular expres-

sion. Multiple keywords joined with `+` require all terms to match. The prefix `~` activates full regex syntax. Results include clickable links to inspect or download each indicator.

`searchsource(#)` filters search results to a specific source ID. Can be used alone to browse all indicators in a source.

`searchtopic(#)` filters search results to a specific topic ID.

`searchfield(`*string*`)` restricts search to a metadata field: `code`, `name`, `description`, `source`, `topic`, `note`, or `all` (default). Multiple fields can be separated by semicolons.

`exact` requires an exact indicator code match rather than partial matching.

`detail` displays search results in wrapped block format with full indicator names and topic labels.

`limit(#)` sets the maximum number of results to display (default 20).

`info(`*string*`)` displays detailed metadata for a specific indicator, including its full name, source database, topic classifications, description, methodology notes with clickable URLs, and download commands.

These commands support the paper's central thesis by making indicator *selection* itself a scripted, auditable process. Rather than browsing web portals manually and noting indicator codes on paper, researchers can document their search and selection process as executable Stata commands that can be inspected, replicated, and version-controlled. Verbatim outputs illustrating each discovery command are reported in Appendix **??**–**??**.

3.3 Stored results

`wbopendata` is an r-class command that stores results in `r()`. These stored results are critical for automation: they allow downstream code to programmatically access indicator metadata, construct dynamic graph titles, and build reproducible pipelines without manual intervention.

Indicator codes and variable names. World Bank indicator codes like `SI.POV.DDAY` contain periods, which Stata does not permit in variable names. The command automatically converts indicator codes to Stata-safe variable names by replacing periods with underscores and converting to lowercase: `SI.POV.DDAY` becomes `si_pov_dday`. Both forms are stored: `r(indicator#)` preserves the original API code for documentation and re-querying, while `r(varname#)` provides the Stata variable name for use in analysis commands.

Indexed versus aggregate returns. Results come in two forms. Indexed returns (`r(varname1)`, `r(varname2)`, ...) store metadata for each indicator separately, enabling indicator-specific labeling and citation. Aggregate returns store combined information: `r(indicator)` contains the full semicolon-separated query string as entered, while `r(name)` contains all variable names as a space-separated list suitable for `foreach`

loops or variable lists.

For each requested indicator (indexed by # = 1, 2, ...), the command returns:

Table 2: Stored results: core command

Result	Type	Description
Aggregate returns (always)		
`r(indicator)`	local	Full query string (semicolon-separated)
`r(name)`	local	All Stata variable names (space-separated)
Indexed returns (per indicator, always)		
`r(indicator#)`	local	Original API indicator code (e.g., SI.POV.DDAY)
`r(varname#)`	local	Stata-safe variable name (e.g., si_pov_dday)
`r(varlabel#)`	local	Indicator label from API
`r(source#)`	local	Source database identifier
`r(time#)`	local	Time dimension name
`r(sourcecite#)`	local	Clean organization name (when Note is non-empty)
With `year()` option		
`r(year#)`	local	Year or year range requested
With `latest` option		
`r(latest)`	local	Formatted subtitle string for graphs
`r(latest_ncountries)`	local	Number of countries with data
`r(latest_avgyear)`	local	Average year of observations
`r(latest_year)`	local	Maximum year retained
With `linewrap()` option		
`r(name#_stack)`	local	Wrapped name for `title()`
`r(description#_stack)`	local	Wrapped description for captions
`r(note#_stack)`	local	Wrapped methodological notes
`r(source#_stack)`	local	Wrapped source text
`r(topic#_stack)`	local	Wrapped topic name
`r(*#_nlines)`	scalar	Line count for each field
`r(*#_line1)`, ...	local	Individual wrapped lines

Discovery commands (Section **??**) return a separate set of results documented in Table **??**.

These returns enable fully automated workflows: a script can download data, extract `r(name1_stack)` for the graph title, `r(sourcecite1)` for the source note, and `r(latest)` for a coverage subtitle—all without hardcoding any metadata. Discovery returns further support automation by enabling scripts to programmatically enumerate available indicators, filter by source or topic, and extract metadata for selected series.

Table 3: Stored results: discovery commands (v18.0)

Result	Type	Description
sources		
r(n_sources)	scalar	Total number of data sources
r(n_indicators)	scalar	Total indicators across all sources
r(source_codes)	local	Space-separated list of source IDs
r(source_names)	local	Compound-quoted list of source names
alltopics		
r(n_topics)	scalar	Total number of topics
r(topic_ids)	local	Space-separated list of topic IDs
r(topic_names)	local	Compound-quoted list of topic names
search(pattern)		
r(n_results)	scalar	Total matches found
r(n_displayed)	scalar	Matches shown after limit
r(first_code)	local	First matching indicator code
r(codes)	local	Space-separated indicator codes
r(keyword)	local	Search keyword used
info(code)		
r(indicator)	local	Indicator code
r(name)	local	Indicator name
r(source_id)	local	Source database ID
r(description)	local	Full indicator description
r(note)	local	Methodology note
All discovery commands		
r(cmd)	local	Reproducible command string

4 Reproducible analytical workflows

This section presents three canonical workflows that illustrate how `wbopendata` supports reproducible empirical research. Rather than exhaustively documenting every option, the workflows are selected to demonstrate the command's core design principles: scripted data acquisition, metadata-driven annotation, and integration into larger analytical pipelines. Extended examples and visualizations appear in Appendix **??**, with diagnostics and QA artifacts in Appendices **??** and **??**.

4.1 Scripted data acquisition and data shape

The most fundamental workflow enabled by `wbopendata` is the scripted retrieval of development indicators with explicit control over data shape. The following command retrieves a single indicator for all countries and returns the data in long format:

```
. wbopendata, indicator(NY.GDP.MKTP.CD) long clear
```

This single line constitutes a complete, executable specification of data provenance. The indicator definition, source database, country coverage, and time dimension are all determined by the API query and documented through the command's stored results. Unlike manual downloads, no implicit filtering or preprocessing occurs outside the script.

The choice between wide and long formats is not cosmetic. Long format aligns naturally with Stata's panel data commands, regression routines, and `bysort` operations, while wide format may be preferable for descriptive or tabular summaries. By making this choice explicit at the point of data acquisition, `wbopendata` ensures that downstream analysis remains transparent and reproducible. Full metadata output associated with this query is reported in Appendix **??**.

4.2 Metadata-driven visualization

A second canonical workflow demonstrates how metadata retrieved programmatically can be used directly to produce publication-ready visualizations. The `latest` and `linewrap()` options enable the command to return machine-readable metadata suitable for graph titles, captions, and source annotations.

The example below retrieves two indicators, retains the most recent comparable observation for each country, and prepares wrapped metadata for use in figures:

```
. wbopendata, indicator(SI.POV.DDAY; SH.DYN.MORT) ///
    long latest linewrap(name description note)
```

The command stores formatted strings in `r()` that can be passed directly to Stata's graphing commands. Figure **??** illustrates this workflow by constructing a scatter plot of poverty headcount ratios against under-five mortality rates, with axis titles, definitions, and source notes populated automatically from the returned metadata.

By treating metadata as structured output rather than narrative text, `wbopendata`

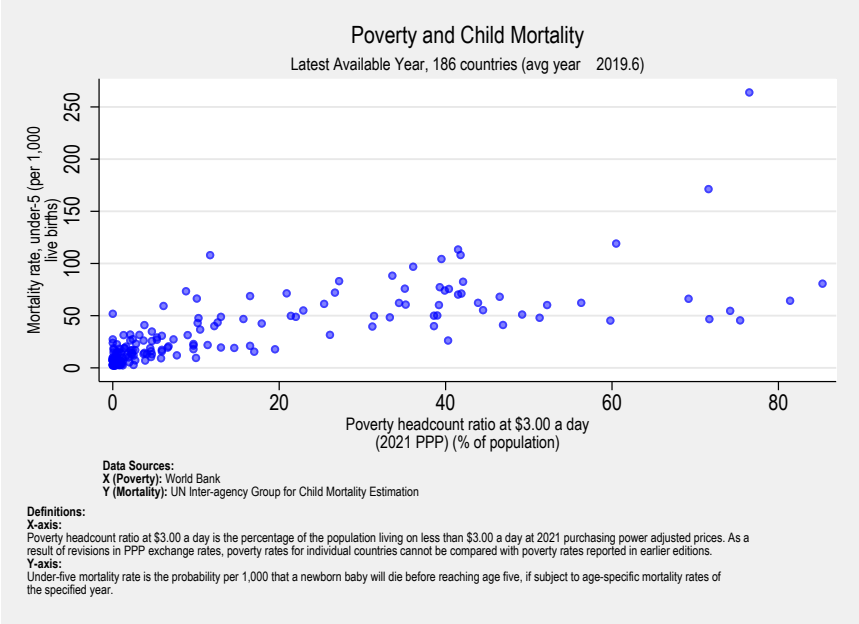

Figure 1: Poverty and child mortality scatter plot with automatic metadata annotation. Axis titles, definitions, and source citations are populated directly from `wbopendata`'s stored results using the `linewrap()` option.

Note: See Appendix **??** for the script used to generate this figure.

reduces transcription errors and ensures consistency between data and documentation. In particular, the `latest` option records coverage diagnostics—including the number of countries and the average observation year—that can be displayed transparently in figure subtitles. Alternative formatting options and full metadata returns are documented in Appendix **??**.

4.3 Integration into analytical pipelines

Beyond single-command use, `wbopendata` is designed to function as infrastructure within larger analytical pipelines. A prominent example is the World Bank's Learning Poverty project (**?**), which computes global and regional estimates of the share of children unable to read and understand a short text by age ten.

In this pipeline, `wbopendata` is used to retrieve population weights, enrollment rates, and auxiliary indicators from the World Development Indicators database. These inputs are combined with harmonized learning assessment data in a fully scripted workflow released under an open-source license with multiple versioned releases. Because all external data dependencies are accessed programmatically, the resulting estimates can be independently replicated and systematically updated as new data become available.

This use case illustrates how the design principles described in Section **??** scale beyond exploratory analysis. By constraining data acquisition to explicit, auditable commands, `wbopendata` enables complex, multi-step analytical systems to remain reproducible over time, even as underlying data sources are revised. Additional pipeline examples and related visualizations are provided in Appendix **??**.

4.4 Indicator discovery as auditable selection

A fourth canonical workflow, introduced in version 18.0, demonstrates how discovery commands make indicator *selection* itself a scripted and auditable process. In traditional workflows, researchers browse web portals, scan indicator lists visually, and record codes manually—a process that leaves no executable trace. Discovery commands replace this with a reproducible sequence:

```
.     * 1. Browse available sources
.     wbopendata, sources
.
.     * 2. Explore a specific source (World Development Indicators = 2)
.     wbopendata, searchsource(2) limit(30)
.
.     * 3. Search for indicators of interest
.     wbopendata, search(poverty) searchtopic(11)
.
.     * 4. Get detailed info on a specific indicator
.     wbopendata, info(SI.POV.DDAY)
.
.     * 5. Download the data
.     wbopendata, indicator(SI.POV.DDAY) clear long
```

Each step in this sequence is an executable Stata command that can be preserved in a do-file, version-controlled, and rerun by independent researchers. The discovery workflow thus extends the principle of data acquisition as code (Section **??**) upstream: not only is the data *download* scripted, but the process of *finding and selecting* indicators is documented as well. Stored results from each discovery command (Section **??**) enable programmatic enumeration of available indicators, filtering by source or topic, and extraction of metadata for selected series—supporting fully automated pipelines from discovery through analysis.

5 Technical implementation and reliability

This section documents the technical implementation of `wbopendata` and the mechanisms used to ensure reliability over time. The focus is deliberately operational: how the command is installed, how it interacts with upstream data infrastructure, and how stability is maintained as external APIs evolve.

5.1 Installation

The recommended installation method is via the Statistical Software Components (SSC) archive:

```
. ssc install wbopendata, replace
```

For users who require the latest development version, including recently added metadata-handling features, the package can be installed directly from the public repository:

```
. net install wbopendata, ///
  from("https://raw.githubusercontent.com/jpazvd/wbopendata/main/src") replace
```

Both methods rely exclusively on Stata's native package management system and require no external dependencies.

5.2 Architecture

`wbopendata` is implemented entirely in Stata's ado-file language and communicates with the World Bank's REST API endpoints (**?**). Key architectural features include automatic pagination (retrieval and assembly of multi-page API responses), deterministic caching using hashed request parameters, and systematic normalization of indicator codes into legal Stata variable names.

Metadata parsing and formatting build on established community utilities, including `tknz` (**?**), `linewrap` (**?**), and `_pecats` (**?**). These components support string tokenization, metadata wrapping, and category handling without introducing external binaries.

Version 18.0 refactors the implementation into a modular architecture of 40 ado

files, with sub-routines (prefixed `__wbod_`) handling YAML parsing, search, sync, cache management, and discovery commands. As of version 18.4, all internal helpers adopt the `__wbod_` namespace (double underscore prefix) to comply with Stata Journal naming conventions and avoid potential conflicts with official Stata routines. This modular design improves maintainability and enables independent testing of components.

The implementation avoids third-party executables, ensuring compatibility with institutional computing environments that restrict software installation. Network requests respect Stata's proxy configuration options (e.g., `set httpproxy on`), allowing the command to operate behind corporate firewalls.

Characteristic metadata (v18.1)

Beginning with version 18.1, `wbopendata` embeds provenance and indicator metadata directly into the `.dta` file using Stata's variable characteristics (`char`) mechanism. This approach follows the precedent established by `freduse` (`?`), which stores per-variable metadata as characteristics attached to each retrieved series.

Two layers of characteristics are written by default. At the dataset level, `_dta` characteristics record the command version, timestamp, username, and the exact syntax used to generate the dataset. At the variable level, each indicator variable carries characteristics for its source database, description, methodology notes, citation, and topic classifications.

This design produces self-documenting datasets: a `.dta` file carries enough metadata to identify what was requested, when, and by whom, without requiring access to the original do-file or log. Because characteristics persist through `save`/`use` cycles, this provenance information travels with the data even when shared across teams or archived.

The characteristic layer complements rather than replaces the existing metadata channels. Do-file syntax documents the researcher's intent; `r()` return values support ephemeral programmatic workflows; and `char` values provide persistent, machine-readable metadata embedded in the output file. Users who prefer not to attach characteristics can suppress them with the `nochar` option.

5.3 Country attributes and classification

When invoked with the `full` option, `wbopendata` appends a set of 17 country attributes that support classification, merging, and stratified analysis. Table **??** summarizes these variables.

These attributes can also be merged into existing datasets using the `match(varname)` option, where `varname` contains World Bank country codes. This enables enrichment without repeated data downloads.

Regional aggregates are identified explicitly via the `region` variable, allowing users to include or exclude aggregate observations using transparent filters (e.g., `keep if`

Table 4: Country attributes returned with `full` option

Variable	Description
`countrycode` / `countryname`	ISO3 code and name
`region` / `regionname`	Region code and name
`adminregion` / `adminregionname`	Administrative region
`incomelevel` / `incomelevelname`	Income classification
`lendingtype` / `lendingtypename`	Lending type (IBRD, IDA, Blend)
`capital`	Capital city name
`latitude` / `longitude`	Capital coordinates

`region != "NA"` for individual countries).

5.4 Testing and error handling

To ensure reliability, `wbopendata` ships with a two-layer validation framework comprising 163 tests that enforce distinct but complementary invariants. The testing philosophy draws on the software certification principles of **?**; the full framework is documented in Appendix **??**.

Layer 1: Feature validation (`qa/run_tests.do`). The primary QA suite exercises the command's features end-to-end against the live World Bank API, covering indicator retrieval, pagination, caching behavior, option handling, and fixes for historical issues. This layer answers the question: *does the code work?*

Layer 2: Documentation validation (`tests/test_help_examples.do`). A second suite validates that every example documented in the help file executes without error, ensuring that documentation accuracy tracks code changes. This layer answers a different question: *does the documentation match the code?*

Both layers share a common test harness pattern (explicit execution, return-code inspection, pass/fail attribution), produce timestamped logs, and enforce a version guard that prevents stale tests from running against newer code. Together they ensure that neither features nor documentation can drift without detection.

Because Stata lacks the mature continuous integration and mocking frameworks available in other statistical environments, the test design prioritizes detection of upstream regressions such as schema changes, pagination failures, or API endpoint deprecations. Layer 1 comprises 92 integrated tests distributed across 15 categories, covering environment validation, data download modes, output formatting, country metadata, regression protection, linewrap graph annotation, update commands, topic queries, language options, population projections, cache management, metadata synchronization, discovery commands, characteristic metadata verification, error condition validation, extreme input cases, and deterministic offline testing. Layer 2 adds 71 tests organized along the same functional categories; ten of the fifteen categories are exercised by both layers, while five (ENV, REG, ERR, EXT, DET) are tested only in Layer 1, reflect-

ing infrastructure and error-path validation that falls outside help-file documentation (Table **??**).

Table 5: Test suite composition (163 tests across two validation layers)

Abbr.	Category	L1	L2	Focus
ENV	Environment	5	–	Network, API, YAML file presence
DISC	Discovery	10	26	Sources, topics, search, filter, info
DL	Download modes	5	14	Indicator, country, multi-indicator
FMT	Output format	3	2	Long format, year range, latest
CTRY	Country options	10	8	Match, full, geo, ISO, regions, income
TOPIC/LANG	Topics & language	2	1	Topic download, Spanish metadata
LW	Linewrap	4	7	Metadata wrapping, formats, scalars
CHAR	Characteristics	6	2	`char` metadata, persistence, `nochar`
Advanced	Advanced features	6	2	Projection, nobasic, nometadata, date
CACHE/SYNC	Cache & sync	13	6	Cache lifecycle, sync, cross-validation
UPD	Maintenance	6	3	Legacy update, describe, sync interop
REG	Regression	4	–	Historical bug fixes (#33, #45, #46, #51)
ERR	Error conditions	8	–	Syntax validation, API errors, empty
EXT	Extreme cases	4	–	Minimal queries, large datasets
DET	Deterministic	6	–	Offline fixtures, value pinning
		92	71	

Note: L1 = feature validation (`qa/run_tests.do`); L2 = documentation validation (`tests/test_help_examples.do`). Categories marked "–" are tested only in L1. See Appendix **??** for full test definitions.

When an indicator code is invalid or deprecated, `wbopendata` returns informative diagnostics that guide users toward corrective action. Verbatim examples of missing indicators, archived series, and update operations are reported in Appendix **??**.

5.5 Metadata management

Regular metadata updates are particularly important when using the `match()` option, as income groups, regional classifications, and even country names may be revised over time. By making metadata management explicit, `wbopendata` reduces the risk of silently propagating outdated classifications into downstream analysis.

In addition to country classifications and descriptive fields, the set of available indicator codes itself evolves over time. Indicators may be deprecated, archived, or reclassified as source databases are revised, merged, or retired, and metadata associated with existing series may be corrected or updated. `wbopendata` treats these changes as first-class events rather than silent failures. Deprecated indicators trigger explicit diagnostic messages that identify the archival location, while revised metadata is propagated through the local cache when updates are applied.

By making indicator availability and metadata revisions explicit, the command reduces the risk that analytical pipelines continue to rely unknowingly on obsolete series or outdated definitions. Verbatim examples illustrating deprecated indicators and metadata updates are reported in Appendix **??**.

5.6 YAML metadata architecture (v18.0)

Version 18.0 introduces a structured metadata layer that supports the discovery commands described in Section ??. The metadata catalog is stored in YAML files within a local cache directory, enabling offline catalog browsing without network access.

Two primary YAML files constitute the metadata catalog:

`_wbopendata_indicators.yaml` contains metadata for all indicators accessible through the World Bank API—over 29,000 records spanning 71 data sources. Each record includes the indicator code, name, source database, topic classifications, description, and methodology notes. At approximately 17 MB and 100,000+ lines, this file is the largest component of the local cache.

`_wbopendata_parameters.yaml` stores country metadata (296 countries and aggregates), source definitions, and topic classifications. This file supports the `sources` and `alltopics` discovery commands and provides the lookup tables used when enriching downloaded data with country attributes.

The cache directory also maintains version-tracking files that record the schema version, last synchronization timestamp, sync method, and a cumulative history of all sync operations. This audit trail allows users and automated pipelines to verify that the metadata driving indicator selection corresponds to a known, documented state.

Performance

Parsing a 17 MB YAML file imposes a non-trivial startup cost. On first invocation, discovery commands require approximately 21 seconds to parse the indicators file and build an in-memory frame cache. Subsequent calls within the same Stata session retrieve results from the cached frame in under 0.5 seconds (Stata 16+). This frame-caching strategy ensures that interactive workflows involving repeated searches, source browsing, or metadata inspection remain responsive after the initial parse.

5.7 Metadata synchronization

The `sync` family of commands (documented in Section ??) provides version-controlled metadata updates with an explicit safety gate: the default `sync` is a dry run that previews changes, while `sync replace` applies them. Three auxiliary commands—`checkupdate`, `cacheinfo`, and `clearcache`—provide version queries and cache housekeeping. For backward compatibility, the legacy aliases `syncforce`, `syncpreview`, and `syncdryrun` remain functional with deprecation warnings, as does `metadataoffline`, which is superseded by the YAML metadata architecture and the discovery commands described in Section ??.

The sync architecture attempts three download pathways in order of preference: a Python-based canonical pipeline (when Python is available), a pure-Stata fallback, and direct download from the GitHub releases page. This layered strategy maximizes com-

patibility across institutional computing environments with varying software restrictions while ensuring that at least one pathway succeeds. Verbatim outputs of sync diagnostics and cache management commands are reported in Appendix **??**–**??**.

5.8 Data response cache (v18.2+)

Version 18.2 introduces a local data response cache that stores API responses on disk, keyed by a hash of the request parameters (indicator codes, country filters, year range, and source identifier). When a subsequent query matches a cached entry whose age is within the configured time-to-live, the command returns the stored response without contacting the World Bank API. This eliminates redundant network traffic during iterative analysis and reduces sensitivity to transient API outages.

The cache time-to-live defaults to seven days and can be adjusted per query with the `cachedays(#)` option. Three housekeeping commands provide explicit cache control: `cleardatacache` removes all cached response files; `resetdatacache` marks all entries as expired without deleting files, so that the next query re-fetches fresh data while preserving the old responses for inspection; and `nocache` bypasses the cache for a single query. The `verbose` option displays cache hit/miss status and file paths during execution.

Beginning with version 18.3, discovery commands (`search`, `info`, `sources`, `alltopics`) resolve metadata from cached YAML files when the metadata cache is current, avoiding API calls entirely for catalog browsing operations. This cache-aware discovery layer further reduces network dependency during interactive workflows.

6 Discussion: reproducibility and constraints in the age of AI

Fifteen years after its initial release, `wbopendata` remains relevant not because of any single technical feature, but because of the methodological principles embedded in its design. As discussed in Section **??**, these principles treat data acquisition as executable infrastructure rather than an informal preprocessing step. This section reflects on the implications of that design in the context of contemporary empirical research, particularly as analytical workflows are increasingly mediated by AI-assisted tools.

6.1 Reproducibility beyond statistical methodology

The reproducibility crisis in the social sciences has been widely documented (**??**). Much of the resulting reform agenda has focused on statistical practice: pre-registration, replication studies, robustness checks, and disclosure of code. While these efforts are essential, they address only part of the problem. In many empirical workflows, the construction of the analytical dataset itself remains opaque, relying on manual downloads, undocumented filters, and ad hoc preprocessing that cannot be fully reconstructed from code alone.

By treating data acquisition as code, `wbopendata` addresses this dimension directly. A command specifying indicators, countries, and time ranges constitutes a complete and executable description of how the analytical dataset was assembled. When rerun, the same command either reproduces the original data exactly or yields systematically updated results if upstream sources have been revised. In both cases, the provenance of the data is explicit and auditable. Reproducibility is therefore strengthened not by constraining statistical analysis, but by constraining how data enter the analytical pipeline.

6.2 Constraints as analytical infrastructure

The increasing adoption of AI-assisted tools has altered the balance of effort in empirical research. Tasks that were once time-consuming—coding, visualization, and narrative drafting—can now be performed rapidly. This acceleration, however, amplifies the importance of upstream constraints. When tools can generate plausible-looking statistics, code, and citations, the credibility of empirical work depends increasingly on the verifiability of its inputs.

In this environment, `wbopendata` functions less as a productivity complement and more as a constraint mechanism. By binding analysis to a single, authoritative API and requiring explicit specification of indicators, coverage, and time periods, it limits the space of admissible data inputs. These constraints do not inhibit analysis; rather, they discipline it by ensuring that downstream automation operates on verifiable and well-documented sources. Acceleration is thereby decoupled from fabrication: analytical workflows can scale without relaxing standards of provenance.

This role is consistent with the design principles outlined in Section **??**. Backward compatibility preserves trust across time, domain-specific syntax reduces opportunities for error, and scripted data access makes provenance the default rather than an afterthought. Together, these features position `wbopendata` as infrastructure that shapes how analysis is conducted, rather than merely facilitating it.

6.3 Generalizability and limits

The principles embodied in `wbopendata` are not unique to development indicators or to Stata. As discussed earlier, similar design logic underpins other data-access tools operating on aggregated indicators and microdata, both within and outside the World Bank. Their recurrence suggests that constraining data access through executable specifications is a generalizable response to reproducibility challenges in applied research.

At the same time, important limits remain. Programmatic access does not resolve substantive issues of data quality, coverage gaps, or methodological breaks. Indicators may be outdated, revised, or unavailable for political or technical reasons. Moreover, secure access to confidential microdata continues to require institution-specific governance arrangements that cannot be fully standardized. `wbopendata` does not eliminate these challenges; it makes them visible by exposing metadata, coverage diagnostics, and

revision histories directly to the user.

6.4 Implications for open science

The experience of `wbopendata` over fifteen years underscores a broader lesson for open science: principles alone are insufficient without infrastructure that embeds them into routine practice. Transparency and reproducibility are widely endorsed norms, yet they are often undermined by tooling that leaves critical steps implicit. By constraining data acquisition through executable commands, tools like `wbopendata` shift reproducibility from aspiration to default behavior.

In this sense, the contribution of `wbopendata` is methodological rather than technological. It demonstrates that disciplined data access can coexist with flexible analysis, and that constraining the earliest stages of the workflow can enhance, rather than limit, analytical innovation. As empirical research continues to evolve in the presence of increasingly powerful automation, such constraint-based infrastructure will remain essential to maintaining credibility and trust.

7 Conclusion

Fifteen years after the World Bank Open Data Initiative transformed development statistics into a global public good, `wbopendata` continues to function as core infrastructure for reproducible empirical research in Stata. The command provides stable, programmatic access to over 29,000 development indicators from 71 databases, encapsulating API communication, pagination, caching, and metadata management within a single, backward-compatible interface.

This longevity reflects deliberate design choices rather than technical inertia. By maintaining backward compatibility across upstream changes, encapsulating complexity behind domain-specific syntax, and treating data acquisition as code rather than manual preprocessing, `wbopendata` shifts reproducibility from an aspirational norm to a default behavior. Versions 18.0 through 18.4 extend this approach with offline discovery commands for catalog browsing and indicator search, a YAML-backed metadata architecture, version-controlled metadata synchronization, publication-ready metadata handling, self-documenting datasets through variable characteristics, a local data response cache with configurable time-to-live, deterministic offline testing, and a two-layer validation framework with 163 tests across feature and documentation validation.

The command's pure-Stata implementation ensures compatibility with institutional computing environments, while the bundled live test suite and QA framework demonstrate how reliability can be sustained even when upstream data systems evolve. More broadly, the experience documented here illustrates that reproducible analytics depends not only on statistical methods, but on disciplined data-access infrastructure that makes provenance explicit, testable, and auditable.

In the age of AI-assisted research, this constraint-based approach is increasingly im-

portant. As analytical workflows accelerate and the cost of producing plausible statistics and narratives falls, credibility hinges on limiting the space of admissible inputs to authoritative, verifiable sources. `wbopendata` illustrates how such constraints can be embedded directly into routine research practice—supporting scale and automation without relaxing standards of trust.

While developed in Stata and applied to World Bank data, the principles distilled from fifteen years of use—scripted data acquisition, conservative evolution, and operational validation—are not tool-specific. They point toward a broader lesson for empirical research and open science: reproducibility is sustained not by documentation alone, but by infrastructure that enforces it upstream.

Acknowledgments

The author thanks Kit Baum for comments on an earlier draft and for pointing toward important references, and Aart Kraay for questions and nudges toward interesting features to be included. The author is grateful to the World Bank colleagues who have built and supported the Open Data APIs over the years, in particular Malarvizhi Veerappan, Rochelle Glenene O'Hagan, Timothy Grant Herzog, and Ana Florina Pirlea. Thanks also to the World Bank Open Data Initiative for making development data freely accessible, and to the many users who have contributed bug reports and feature suggestions through GitHub.

About the authors

João Pedro Azevedo is Deputy Director and Chief Statistician in UNICEF's Division of Data, Analytics, Planning and Monitoring. Previously, he worked for 16 years at the World Bank as Lead Economist. His research focuses on poverty measurement, education statistics, and reproducible and scalable analytical pipelines for global, regional, and national monitoring systems to inform policy.

The software repository (https://github.com/jpazvd/wbopendata) includes example do-files, test scripts, and complete documentation.

A Extended examples and visualizations

This appendix provides extended examples, alternative visualizations, and verbatim output referenced in the main text. These materials are included to support full transparency and reproducibility while keeping the exposition in Section **??** focused on canonical analytical workflows.

A.1 Extended data retrieval examples

This subsection reports additional data retrieval examples illustrating variations in indicator selection, output shape, and filtering options. These examples expand on the

scripted data acquisition workflow described in Section **??**.

Single-indicator retrieval (verbatim output)

```
. wbopendata, indicator(NY.GDP.MKTP.CD) clear linewrap(name note) maxlength(35 7
      0)

    Metadata for indicator NY.GDP.MKTP.CD
-------------------------------------------------------------------------------
    Name: GDP (current US$)
-------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
-------------------------------------------------------------------------------
    Description: Gross domestic product is the total income earned through
    the production of goods and services in an economic territory during an
    accounting period. It can be measured in three different ways: using
    either the expenditure approach, the income approach, or the production
    approach. This indicator is expressed in current prices, meaning no
    adjustment has been made to account for price changes over time. This
    indicator is expressed in United States dollars.
-------------------------------------------------------------------------------
    Note: Country official statistics, National Statistical Organizations
    and or Central Banks;
-------------------------------------------------------------------------------
    Topic(s):  ; 3 Economy and Growth
```

Multiple-indicator queries (verbatim output)

The following example retrieves multiple indicators simultaneously and returns the data in long format:

```
. wbopendata, indicator(SI.POV.DDAY;NY.GDP.PCAP.PP.KD) clear long ///
      linewrap(name note) maxlength(35 70)

    Metadata for indicator SI.POV.DDAY
-------------------------------------------------------------------------------
    Name: Poverty headcount ratio at $3.00 a day (2021 PPP) (% of
    population)
-------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
-------------------------------------------------------------------------------
    Description: Poverty headcount ratio at $3.00 a day is the percentage of
    the population living on less than $3.00 a day at 2021 purchasing power
    adjusted prices. As a result of revisions in PPP exchange rates, poverty
    rates for individual countries cannot be compared with poverty rates
    reported in earlier editions.
-------------------------------------------------------------------------------
    Note: World Bank, Poverty and Inequality Platform. Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database. For more
    information and methodology, please see http://pip.worldbank.org., World
    Bank (WB), uri: http://pip.worldbank.org, note: Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database.
-------------------------------------------------------------------------------
```

```
    Topic(s): 11 Poverty ; 2 Aid Effectiveness ; 19 Climate Change
-------------------------------------------------------------------------------

    Metadata for indicator NY.GDP.PCAP.PP.KD
-------------------------------------------------------------------------------
    Name: GDP per capita, PPP (constant 2021 international $)
-------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
-------------------------------------------------------------------------------
    Description: This indicator provides values for gross domestic product
    (GDP) expressed in constant international dollars, converted by
    purchasing power parities (PPPs). PPPs account for the different price
    levels across countries and thus PPP-based comparisons of economic
    output are more appropriate for comparing the output of economies and
    the average material well-being of their inhabitants than exchange-rate
    based comparisons.
-------------------------------------------------------------------------------
    Note: .
-------------------------------------------------------------------------------
    Topic(s):
-------------------------------------------------------------------------------

. describe

 Observations:        17,290
    Variables:            13                  5 Jan 2026 14:06
-------------------------------------------------------------------------------
Variable      Storage   Display    Value
    name         type    format    label      Variable label
-------------------------------------------------------------------------------
countrycode     str3    %9s                   Country Code
countryname     str75   %75s                  Country Name
region          str3    %9s                   Region Code
regionname      str51   %51s                  Region Name
adminregion     str3    %9s                   Administrative Region Code
adminregionname str75   %75s                  Administrative Region Name
incomelevel     str3    %9s                   Income Level Code
incomelevelname str19   %19s                  Income Level Name
lendingtype     str3    %9s                   Lending Type Code
lendingtypename str14   %14s                  Lending Type Name
year            int     %9.0g                 Year
si_pov_dday     float   %9.0g                 SI.POV.DDAY
ny_gdp_pcap_p~d float   %9.0g                 NY.GDP.PCAP.PP.KD
-------------------------------------------------------------------------------
Sorted by: countrycode  year
     Note: Dataset has changed since last saved.
```

This pattern is commonly used in multivariate analysis and panel regressions, where alignment across indicators and years is required.

Country- and topic-based queries

Additional examples demonstrate retrieval by country codes and topic identifiers, illustrating how `wbopendata` supports exploratory analysis and bulk downloads without manual file handling.

```
wbopendata, country(BRA; ARG; CHL) long clear
```

```
wbopendata, topics(11) clear
```

Full metadata output associated with these queries is reported in Appendix **??** and Appendix **??**.

A.2 Extended metadata output

This subsection reports verbatim metadata returned by `wbopendata`, including indicator names, definitions, sources, and topic classifications. These outputs correspond to examples discussed in Sections **??** and **??**.

Indicator metadata (illustrative excerpt)

```
Metadata for indicator SI.POV.DDAY
-----------------------------------------------------------
Name: Poverty headcount ratio at $3.00 a day (2021 PPP)
Description: ...
Source: World Bank, Poverty and Inequality Platform
Topic(s): Poverty
-----------------------------------------------------------
```

Metadata is retrieved directly from the World Bank API and cached locally to ensure consistency across repeated queries.

Coverage diagnostics with `latest` (verbatim output)

The following output illustrates coverage diagnostics returned by the `latest` option, including the number of countries and the average observation year:

```
. wbopendata, indicator(SI.POV.DDAY) clear long latest ///
      linewrap(name note) maxlength(35 70)

    Metadata for indicator SI.POV.DDAY
-------------------------------------------------------------------------------
    Name: Poverty headcount ratio at $3.00 a day (2021 PPP) (% of
    population)
-------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
-------------------------------------------------------------------------------
    Description: Poverty headcount ratio at $3.00 a day is the percentage of
    the population living on less than $3.00 a day at 2021 purchasing power
    adjusted prices. As a result of revisions in PPP exchange rates, poverty
    rates for individual countries cannot be compared with poverty rates
    reported in earlier editions.
-------------------------------------------------------------------------------
    Note: World Bank, Poverty and Inequality Platform. Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database. For more
    information and methodology, please see http://pip.worldbank.org., World
```

```
    Bank (WB), uri: http://pip.worldbank.org, note: Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database.
-------------------------------------------------------------------------------
    Topic(s): 11 Poverty ; 2 Aid Effectiveness ; 19 Climate Change
-------------------------------------------------------------------------------

. di as text "latest year: "
latest year: 2024

. di as text "countries: "
countries: 186

. di as text "avg year: "
avg year:    2019.8
```

`linewrap()` stored results (verbatim output)

This output shows the wrapped metadata and stored results used for metadata-driven graph annotation:

```
. * Download indicators with linewrap option for graph-ready metadata
. * Returns r(name#_stack), r(description#_stack), r(sourcecite#), r(latest)
. wbopendata, indicator(SI.POV.DDAY; SH.DYN.MORT) clear long latest ///
      linewrap(name description note) maxlength(40 160)

    Metadata for indicator SI.POV.DDAY
-------------------------------------------------------------------------------
    Name: Poverty headcount ratio at $3.00 a day (2021 PPP) (% of
    population)
-------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
-------------------------------------------------------------------------------
    Description: Poverty headcount ratio at $3.00 a day is the percentage of
    the population living on less than $3.00 a day at 2021 purchasing power
    adjusted prices. As a result of revisions in PPP exchange rates, poverty
    rates for individual countries cannot be compared with poverty rates
    reported in earlier editions.
-------------------------------------------------------------------------------
    Note: World Bank, Poverty and Inequality Platform. Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database. For more
    information and methodology, please see http://pip.worldbank.org., World
    Bank (WB), uri: http://pip.worldbank.org, note: Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database.
-------------------------------------------------------------------------------
    Topic(s): 11 Poverty ; 2 Aid Effectiveness ; 19 Climate Change
-------------------------------------------------------------------------------

    Metadata for indicator SH.DYN.MORT
-------------------------------------------------------------------------------
    Name: Mortality rate, under-5 (per 1,000 live births)
-------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
```

```
-------------------------------------------------------------------------------
    Description: Under-five mortality rate is the probability per 1,000 that
    a newborn baby will die before reaching age five, if subject to
    age-specific mortality rates of the specified year.
-------------------------------------------------------------------------------
    Note: UN Inter-agency Group for Child Mortality Estimation, UN
    Children´s Fund (UNICEF), uri: http://www.childmortality.org, publisher:
    UNICEF, WHO, World Bank, United Nations Population Division;
-------------------------------------------------------------------------------
    Topic(s):
-------------------------------------------------------------------------------

. * Display wrapped metadata available for graph annotations
. return list

macros:
              r(name) : "si_pov_dday  sh_dyn_mort"
       r(latest_year) : "2023"
    r(latest_avgyear) : "   2019.6"
  r(latest_ncountries) : "186"
           r(latest) : "Latest Available Year, 186 countries (avg year    .."
        r(indicator) : "SI.POV.DDAY; SH.DYN.MORT"
            r(time2) : "year"
        r(varlabel2) : "Mortality rate, under-5 (per 1,000 live births)"
          r(source2) : "2 World Development Indicators"
       r(indicator2) : "SH.DYN.MORT"
         r(varname2) : "sh_dyn_mort"
      r(sourcecite2) : "UN Inter-agency Group for Child Mortality Estimati.."
      r(note2_stack) : ""UN Inter-agency Group for Child Mortality Estimat.."
  r(description2_stac
    k)               : ""Under-five mortality rate is the probability per .."
      r(name2_stack) : ""Mortality rate, under-5 (per 1,000" "live births)""
           r(time1) : "year"
        r(varlabel1) : "Poverty headcount ratio at $3.00 a day (2021 PPP) .."
          r(source1) : "2 World Development Indicators"
       r(indicator1) : "SI.POV.DDAY"
         r(varname1) : "si_pov_dday"
      r(sourcecite1) : "World Bank"
      r(note1_stack) : ""World Bank, Poverty and Inequality Platform. Data.."
  r(description1_stac
    k)               : ""Poverty headcount ratio at $3.00 a day is the per.."
      r(name1_stack) : ""Poverty headcount ratio at $3.00 a day" "(2021 PP.."
```

A.3 Extended visualizations

This subsection presents additional figures referenced but not reproduced in the main text. These include alternative mappings, color schemes, and regional breakdowns. Where relevant, the corresponding console output and commands are provided verbatim.

Metadata-driven graph annotation (supporting Figure ??)

```
. * Graph with wrapped axis titles, subtitle, definitions, and sources
. set scheme sj

. twoway (scatter sh_dyn_mort si_pov_dday, msize(small) mcolor(blue%50)), ///
      xtitle("Poverty headcount ratio at $3.00 a day" "(2021 PPP) (% of population)", size(small)) ///
```

```
      ytitle("Mortality rate, under-5 (per 1,000" "live births)", size(small)) ///
      title("Poverty and Child Mortality", size(medium)) ///
      subtitle("Latest Available Year, 186 countries (avg year    2019.6)", size(small)) ///
      caption("bf:Definitions:" ///
      "bf:X-axis: " "Poverty headcount ratio at $3.00 a day is the percentage of the population living on less
      "bf:Y-axis: " "Under-five mortality rate is the probability per 1,000 that a newborn baby will die before
      note("bf:Data Sources:" ///
      "bf:X (Poverty): World Bank" ///
      "bf:Y (Mortality): UN Inter-agency Group for Child Mortality Estimation", size(vsmall)) name(tmp1, replac
```

Choropleth mapping example (verbatim output)

```
. * Download indicator data
. tempfile wdi_data
. wbopendata, indicator(it.cel.sets.p2) long clear latest

    Metadata for indicator IT.CEL.SETS.P2
-------------------------------------------------------------------------------
    Name: Mobile cellular subscriptions (per 100 people)
-------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
-------------------------------------------------------------------------------
    Description: Mobile cellular telephone subscriptions are subscriptions
    to a public mobile telephone service that provide access to the PSTN
    using cellular technology. The indicator includes (and is split into)
    the number of postpaid subscriptions, and the number of active prepaid
    accounts (i.e. that have been used during the last three months). The
    indicator applies to all mobile cellular subscriptions that offer voice
    communications. It excludes subscriptions via data cards or USB modems,
    subscriptions to public mobile data services, private trunked mobile
    radio, telepoint, radio paging and telemetry services.
-------------------------------------------------------------------------------
    Note: World Telecommunication ICT Indicators Database, International
    Telecommunication Union (ITU)
-------------------------------------------------------------------------------
    Topic(s): 9 Infrastructure
-------------------------------------------------------------------------------

. sort countrycode
. * Merge with shapefile coordinates
. use "C:/Users/jpazevedo/ado/plus/w/world-d.dta", clear
. * Create choropleth map
. set scheme sj
. sum year

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
        year |        262    2022.584    2.400234       2004       2024
. local avg = string(<avg>, "%16.1f")
. spmap it_cel_sets_p2 using "C:/Users/jpazevedo/ado/plus/w/world-c.dta", id(_ID) ///
      clnumber(20) fcolor(Reds2) ocolor(none ..) ///
      title("Mobile cellular subscriptions (per 100 people)", size(*1.2)) ///
      legstyle(3) legend(ring(1) position(3)) ///
      note("Source: World Telecommunication ICT Indicators Database (latest: 2022.6)")
```

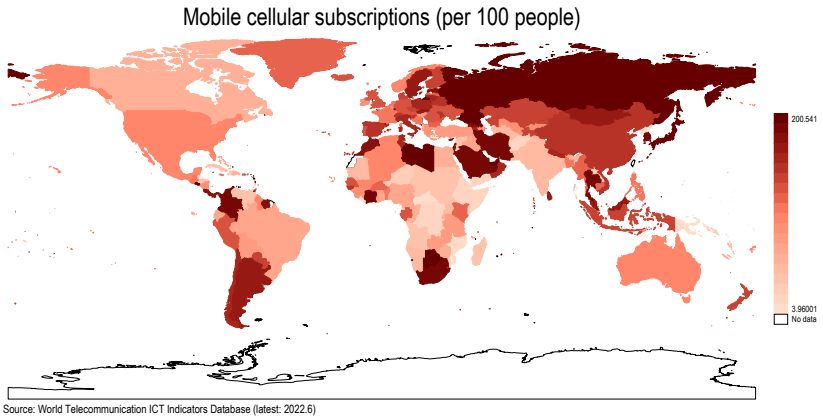

Figure A1: Mobile cellular subscriptions per 100 people (latest available year). Created by merging `wbopendata` output with geographic shape data and visualizing using the `spmap` command (**?**).

Alternative scatter plots (verbatim output)

```
. * Scatter plot: Poverty vs GDP per capita with lowess smoother
. set scheme sj

. graph twoway ///
      (scatter si_pov_dday ny_gdp_pcap_pp_kd, msize(*.3)) ///
      (scatter si_pov_dday ny_gdp_pcap_pp_kd if regionname == "Aggregates", ///
      msize(*.8) mlabel(countryname) mlabsize(*.8) mlabangle(25)) ///
      (lowess si_pov_dday ny_gdp_pcap_pp_kd), ///
      legend(off) ///
      ytitle("Poverty headcount ratio at $2.15 a day", size(small)) ///
      xtitle("GDP per capita, PPP (constant intl $)", size(small)) ///
      note("Source: WDI (latest as of  5 Jan 2026 14:07)")
```

A.4 User-written extensions

This subsection shows examples of user-written commands that rely on `wbopendata` internally.

`worldstat`: Africa example (verbatim output)

```
. * Regional map: GDP per capita in Africa (2009)
. * Options: stat(GDP), year(2009), cname displays country names
. worldstat Africa, stat(GDP) year(2009) cname
worldstat is built using the functionality of the module wbopendata.
checking wbopendata consistency and verifying not already installed...
Accessing shape file for Africa to create geographical visualisation
Accessing shape files for map output remotely
(prefix now "http://damianclarke.net/stata/worldstat")
Importing GDP from World Bank database
```

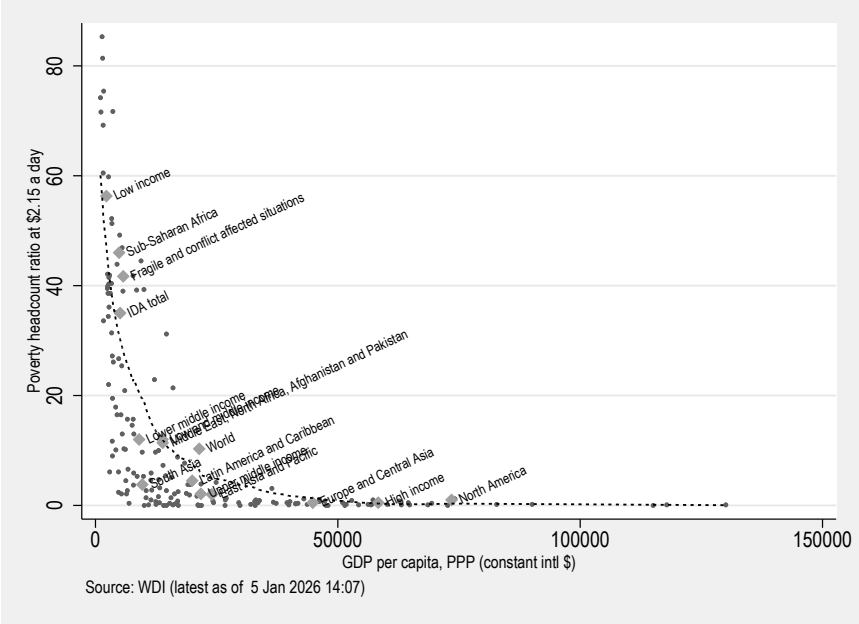

Figure A2: Poverty headcount ratio versus GDP per capita (PPP, constant international $). Regional aggregates are labeled; lowess smoother shows the cross-country relationship.

```
    Metadata for indicator NY.GDP.PCAP.KD
--------------------------------------------------------------------------------
    Name: GDP per capita (constant 2015 US$)
--------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
--------------------------------------------------------------------------------
    Description: Gross domestic product is the total income earned through
    the production of goods and services in an economic territory during an
    accounting period. It can be measured in three different ways: using
    either the expenditure approach, the income approach, or the production
    approach. The core indicator has been divided by the general population
    to achieve a per capita estimate.This indicator is expressed in constant
    prices, meaning the series has been adjusted to account for price
    changes over time. The reference year for this adjustment is 2015. This
    indicator is expressed in United States dollars.
--------------------------------------------------------------------------------
    Note: Country official statistics, National Statistical Organizations
    and or Central Banks;
--------------------------------------------------------------------------------
    Topic(s):  ; 3 Economy and Growth
--------------------------------------------------------------------------------

Visualising data
```

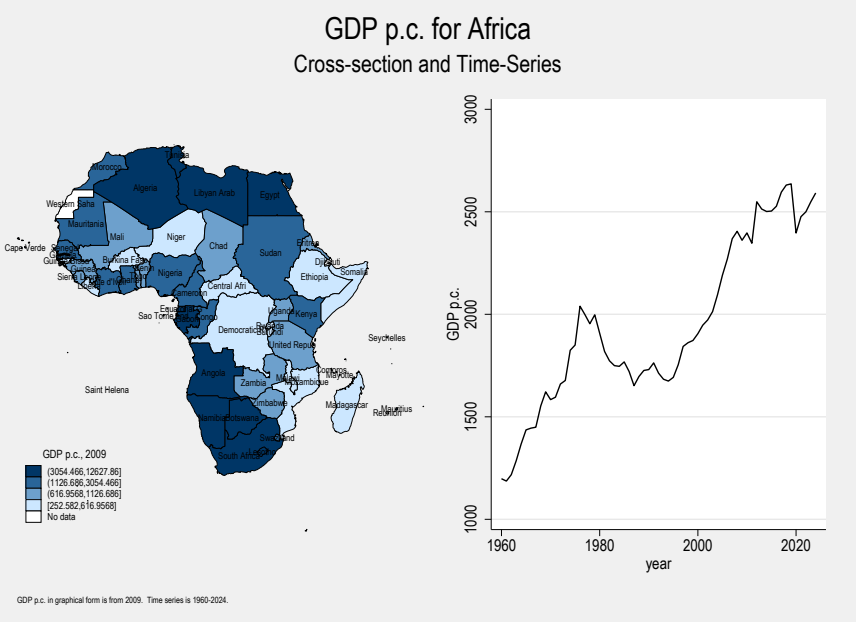

Figure A3: GDP per capita (constant 2015 US$) in Africa, 2009. Generated using `worldstat` (**?**), which calls `wbopendata` internally.

`worldstat`: **global example (verbatim output)**

```
. * Global map: Fertility rate with Pastel2 color scheme
. worldstat world, stat(FERT) fcolor(Pastel2)
worldstat is built using the functionality of the module wbopendata.
checking wbopendata consistency and verifying not already installed...
Accessing shape file for world to create geographical visualisation
Accessing shape files for map output remotely
(prefix now "http://damianclarke.net/stata/worldstat")
Importing FERT from World Bank database

    Metadata for indicator SP.DYN.TFRT.IN
--------------------------------------------------------------------------------
    Name: Fertility rate, total (births per woman)
--------------------------------------------------------------------------------
    Collection: 2 World Development Indicators
--------------------------------------------------------------------------------
    Description: Total fertility rate represents the number of children that
    would be born to a woman if she were to live to the end of her
    childbearing years and bear children in accordance with age-specific
    fertility rates of the specified year.
--------------------------------------------------------------------------------
    Note: World Population Prospects, United Nations (UN), publisher: UN
    Population Division;
--------------------------------------------------------------------------------
    Topic(s):  ; 8 Health
--------------------------------------------------------------------------------

Visualising data
```

A.5 Notes on reproducibility

All examples in this appendix are fully reproducible using the version of `wbopendata` documented in the main text. Because outputs depend on live API responses, results may differ over time as underlying data are revised. Such differences reflect upstream updates rather than changes in analytical logic and can be identified by re-running the corresponding commands.

B Diagnostics and verbatim outputs

B.1 Missing indicator example

```
. wbopendata, language(en) indicator(platypus) long clear

    Sorry... No data was downloaded for indicator platypus.

    (1) Please check your internet connection by clicking here, if does not
    work please check with your internet provider or IT support, otherwise...
    (2) Please check your access to the World Bank API by clicking here, if
    does not work please check with your firewall settings or internet
    provider or IT support, otherwise...
    (3) Please check the availability of your indicator or topic by clicking
    here. If the paramater value is not valid...
```

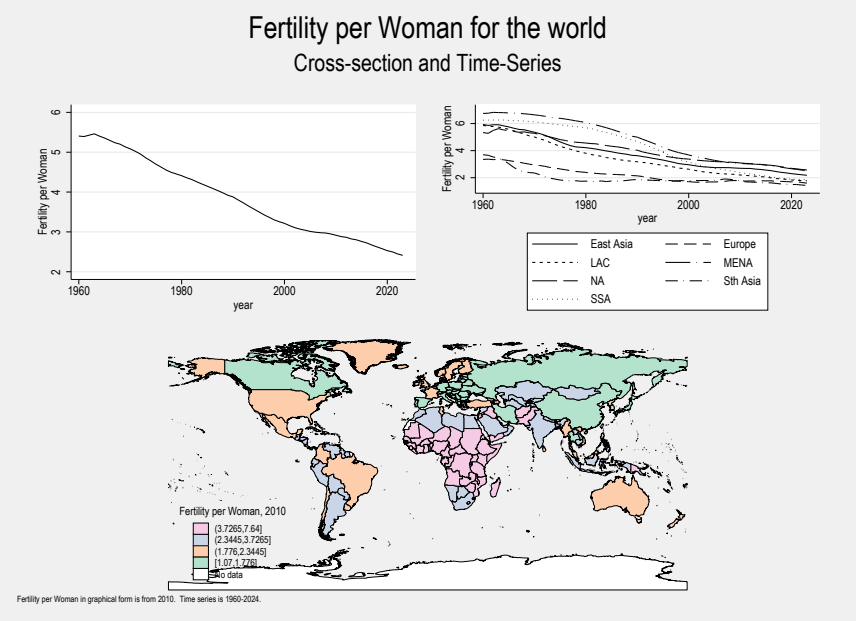

Figure A4: Fertility rate (births per woman) worldwide. The `worldstat` command uses `wbopendata` internally to retrieve World Bank indicators.

```
    (4) Please check the list of available indictator(s) or topic(s) in the
    help wbopendata or by visiting the API query builder, if all the above
    seems fine...
    (5) Please consider ajusting your Stata timeout parameters. For more
    details see netio.
    (6) Please send us an email to report this error by clicking here or
    writing to:
            email: data@worldbank.org
            subject: wbopendata query error at 5 Jan 2026 14:07:20:
    https://api.worldbank.org/v2/en/countries/all/Indicators/platypus?download
    > format=CSV&HREQ=N&filetype=data

. di as text "Captured return code (expected nonzero): "
Captured return code (expected nonzero):
```

B.2 Deprecated / archived indicator example

```
. wbopendata, language(en) indicator(AG.AGR.TRAC.NO) clear

    Sorry... but indicator AG.AGR.TRAC.NO has been moved to 57 WDI Database
    Archives.

    Please send us an email to obtain more information clicking here or
    writing to:
            email: data@worldbank.org
            subject: wbopendata query error 23 [AG.AGR.TRAC.NO - Agricultural
```

```
    machinery, tractors] at 5 Jan 2026 14:07:20:
    https://api.worldbank.org/v2/Indicators/AG.AGR.TRAC.NO

. di as text "Captured return code (expected r(23) archive notice): "
Captured return code (expected r(23) archive notice):
```

B.3 Metadata sync preview output

```
r; t=0.00 3:25:11
. wbopendata, sync
─────────────────────────────────────────────────────────────────────────
wbopendata Metadata Status
─────────────────────────────────────────────────────────────────────────

  Cache Status
  ──────────────────────────────────────────
  Version:          v2.0.0
  Last sync:        9 Feb 2026 14:44:41
  Sync method:      python
  Cached Records
  ──────────────────────────────────────────
  Indicators:          29,243
  Sources:                 71
  Topics:                  21
  Country metadata:       296
  Remote Status (WB API)
  ──────────────────────────────────────────
  API indicators:      29,323
  Change:           +80 new
  GitHub Release
  ──────────────────────────────────────────
  Latest version:   v18.1.0
  Status:           Up to date
  Sync Pathway
  ──────────────────────────────────────────
  Python:           available (Python 3.11.5)
  Will use:         Python canonical
─────────────────────────────────────────────────────────────────────────

Actions:
    Sync metadata now
    Force sync (even if fresh)
    Force Stata pathway
    View sources
    View topics
    Search indicators
─────────────────────────────────────────────────────────────────────────
r; t=4.84 3:25:16
```

B.4 Discovery: Available sources

```
r; t=0.01 3:24:39
. wbopendata, sources
```

```
World Bank Data Sources (71 sources, 29189 indicators)
```

Code	Name	Indicators	[Browse]
1	Doing Business	190	[Browse]
2	World Development Indicators	1504	[Browse]
3	Worldwide Governance Indicators	0	[Browse]
5	Subnational Malnutrition Database	5	[Browse]
6	International Debt Statistics	0	[Browse]
11	Africa Development Indicators	837	[Browse]
12	Education Statistics	8308	[Browse]
13	Enterprise Surveys	115	[Browse]
14	Gender Statistics	800	[Browse]
15	Global Economic Monitor	36	[Browse]
16	Health Nutrition and Population Statistics	163	[Browse]
18	IDA Results Measurement System	1	[Browse]
19	Millennium Development Goals	19	[Browse]
20	Quarterly Public Sector Debt	564	[Browse]
22	Quarterly External Debt Statistics SDDS	1800	[Browse]
23	Quarterly External Debt Statistics GDDS	256	[Browse]
25	Jobs	3	[Browse]
27	Global Economic Prospects	1	[Browse]
28	Global Findex database	2947	[Browse]
29	The Atlas of Social Protection: Indicat...	3561	[Browse]

```
Showing 20 of 71 sources. Use limit(#) to see more.

Tip: Click [Browse] to see all indicators from a source
     Use search(keyword) searchsource(#) to filter within a source
r; t=0.84 3:24:40
```

B.5 Discovery: Indicator search

```
r; t=0.00 3:24:41

. wbopendata, search(poverty) searchtopic(11) limit(10)
(Caching metadata in memory...)

Search results for "poverty" (showing 10 of 145 matches)
```

Code	Name	Source	Topic
1.0.HCount.1.90usd	Poverty Headcount ($1.90 ...	37	Poverty
> [Info] [Get]			
1.0.HCount.2.5usd	Poverty Headcount ($2.50 ...	37	Poverty
> [Info] [Get]			
1.0.HCount.Mid10to50	Middle Class ($10-50 a da...	37	Poverty
> [Info] [Get]			
1.0.HCount.Ofcl	Official Moderate Poverty...	37	Poverty
> [Info] [Get]			
1.0.HCount.Poor4uds	Poverty Headcount ($4 a day)	37	Poverty
> [Info] [Get]			
1.0.HCount.Vul4to10	Vulnerable ($4-10 a day) ...	37	Poverty
> [Info] [Get]			
1.0.PGap.1.90usd	Poverty Gap ($1.90 a day)	37	Poverty
> [Info] [Get]			
1.0.PGap.2.5usd	Poverty Gap ($2.50 a day)	37	Poverty
> [Info] [Get]			
1.0.PGap.Poor4uds	Poverty Gap ($4 a day)	37	Poverty

```
> [Info] [Get]
1.0.PSev.1.90usd       Poverty Severity ($1.90 a...       37 Poverty
> [Info] [Get]
```

```
Showing 10 of 145 results. Use limit(#) to see more.

Click [Info] for details, [Get] to download, Source/Topic to browse similar
r; t=28.66 3:25:09
```

B.6 Discovery: Indicator metadata

```
r; t=0.00 3:25:09

. wbopendata, info(SI.POV.DDAY)
(Using cached metadata from memory)
```

```
    Indicator: SI.POV.DDAY
```

```
    Name: Poverty headcount ratio at $3.00 a day (2021 PPP) (% of
    population)
```

```
    Source ID: 2
```

```
    Source: World Development Indicators
```

```
    Topic ID(s): 11; 2; 19
```

```
    Topic(s): Poverty; Aid Effectiveness; Climate Change
```

```
    Description: Poverty headcount ratio at $3.00 a day is the percentage of
    the population living on less than $3.00 a day at 2021 purchasing power
    adjusted prices. As a result of revisions in PPP exchange rates, poverty
    rates for individual countries cannot be compared with poverty rates
    reported in earlier editions.
```

```
    Note: World Bank, Poverty and Inequality Platform. Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database. For more
    information and methodology, please see http://pip.worldbank.org., World
    Bank (WB), uri: http://pip.worldbank.org, note: Data are based on
    primary household survey data obtained from government statistical
    agencies and World Bank country departments. Data for high-income
    economies are mostly from the Luxembourg Income Study database.
```

```
Filters:
  [searchsource(2)] |   [searchtopic(11)]
```

```
Download:
  [Wide format]
  [Long format]
  [Specific countries]
```

```
r; t=0.14 3:25:10
```

B.7 Discovery: Topic categories

```
r; t=0.00 3:25:10
. wbopendata, alltopics
World Bank Topics (21 categories)
```

ID	Name	Indicators	[Browse]
1	Agriculture & Rural Development	49	[Browse]
2	Aid Effectiveness	77	[Browse]
3	Economy & Growth	306	[Browse]
4	Education	1023	[Browse]
5	Energy & Mining	53	[Browse]
6	Environment	188	[Browse]
7	Financial Sector	203	[Browse]
8	Health	661	[Browse]
9	Infrastructure	78	[Browse]
10	Social Protection & Labor	2042	[Browse]
11	Poverty	145	[Browse]
12	Private Sector	197	[Browse]
13	Public Sector	155	[Browse]
14	Science & Technology	13	[Browse]
15	Social Development	35	[Browse]
16	Urban Development	27	[Browse]
17	Gender	322	[Browse]
18	Millenium development goals	26	[Browse]
19	Climate Change	82	[Browse]
20	External Debt	516	[Browse]
21	Trade	155	[Browse]

```
Tip: Click [Browse] to see all indicators in a topic
     Use search(keyword) searchtopic(#) to filter within a topic
r; t=0.95 3:25:11
```

B.8 Sync: Detailed metadata breakdown

```
r; t=0.00 3:25:16
. wbopendata, sync detail
wbopendata Metadata Status
  Cache Status
  Version:          v2.0.0
  Last sync:        9 Feb 2026 14:44:41
  Sync method:      python
  Cached Records
  Indicators:          29,243
  Sources:                 71
  Topics:                  21
  Country metadata:       296
  Remote Status (WB API)
  API indicators:      29,323
```

```
Change:           +80 new
GitHub Release
──────────────────────────────────────────
Latest version:   v18.1.0
Status:           Up to date
Sync Pathway
──────────────────────────────────────────
Python:           available (Python 3.11.5)
Will use:         Python canonical
```

Indicators by Source

ID	Source Name	Indicators
12	Education Statistics	8,308
29	The Atlas of Social Protection: Indicato	3,561
28	Global Findex database	2,947
22	Quarterly External Debt Statistics SDDS	1,800
92	Disability Data Hub (DDH)	1,577
2	World Development Indicators	1,504
57	WDI Database Archives	975
11	Africa Development Indicators	837
14	Gender Statistics	800
86	Global Jobs Indicators Database (JOIN)	746
34	Global Partnership for Education	678
20	Quarterly Public Sector Debt	564
81	International Debt Statistics: DSSI	532
39	Health Nutrition and Population Statisti	420
87	Country Climate and Development Report (	392
69	Global Financial Inclusion and Consumer	347
65	Health Equity and Financial Protection I	315
64	Worldwide Bureaucracy Indicators	302
45	Indonesia Database for Policy and Econom	266
23	Quarterly External Debt Statistics GDDS	256
37	LAC Equity Lab	211
1	Doing Business	190
41	Country Partnership Strategy for India (	185
16	Health Nutrition and Population Statisti	163
59	Wealth Accounts	146
33	G20 Financial Inclusion Indicators	131
68	PEFA 2016	129
13	Enterprise Surveys	115
82	Global Public Procurement	111
32	Global Financial Development	108
30	Exporter Dynamics Database - Indicator	98
83	Statistical Performance Indicators (SPI)	72
89	Identification for Development (ID4D) Da	71
91	PEFA_CRPFM	43
15	Global Economic Monitor	36
54	Joint External Debt Hub	28
63	Human Capital Index	27
88	Food Prices for Nutrition	23
79	PEFA_GRPFM	21
93	FPN Datahub Archive	20
73	Global Financial Inclusion and Consumer	19
19	Millennium Development Goals	19
80	Gender Disaggregated Labor Database (GDL	16
66	Logistics Performance Index	12
35	Sustainable Energy for All	11

46	Sustainable Development Goals	10
75	Environment, Social and Governance (ESG)	9
58	Universal Health Coverage	8
5	Subnational Malnutrition Database	5
61	PPPs Regulatory Quality	4
67	PEFA 2011	4
78	ICP 2017	3
25	Jobs	3
70	Economic Fitness 2	2
84	Education Policy	2
60	Economic Fitness	2
43	Adjusted Net Savings	2
18	IDA Results Measurement System	1
27	Global Economic Prospects	1
50	Subnational Population	1

Indicators by Topic

ID	Topic Name	Indicators
10	Social Protection & Labor	2,042
4	Education	1,023
8	Health	661
20	External Debt	516
17	Gender	322
3	Economy & Growth	306
7	Financial Sector	203
12	Private Sector	197
6	Environment	188
21	Trade	155
13	Public Sector	155
11	Poverty	145
19	Climate Change	82
9	Infrastructure	78
2	Aid Effectiveness	77
5	Energy & Mining	53
1	Agriculture & Rural Development	49
15	Social Development	35
16	Urban Development	27
18	Millenium development goals	26
14	Science & Technology	13

Actions:

Sync metadata now
Force sync (even if fresh)
Force Stata pathway

View sources
View topics
Search indicators

r; t=2.72 3:25:19

B.9 Cache: Update check

```
r; t=0.00 3:26:20
.     wbopendata, checkupdate
r; t=0.62 3:26:21
.     sjlog close, replace
```

C Quality assurance, testing, and operational validation

This appendix documents the quality assurance (QA) and testing framework used to maintain the reliability of `wbopendata` over time. The framework builds on principles articulated by **?**, who established that statistical software certification requires systematic testing where "tests are never thrown away" and reliability is demonstrated through cumulative evidence rather than single-point verification. These practices are a core component of the command's reproducibility guarantees, ensuring that changes in upstream data infrastructure, metadata, and indicator availability are detected, diagnosed, and addressed explicitly.

Unlike statistical reproducibility, which concerns the stability of estimates conditional on data, QA for data-access infrastructure must address a distinct set of risks: API schema changes, endpoint deprecations, evolving indicator vocabularies, pagination failures, and environment-specific constraints. The framework described here is designed to surface such issues under real-world operating conditions.

C.1 Testing philosophy: operational validation

The testing strategy for `wbopendata` is based on *live integration tests* that exercise the command end-to-end against the World Bank Data API. Tests intentionally rely on live network access and real data downloads, rather than mocked responses or static fixtures.

This approach differs from the release-gate testing paradigms used by CRAN or PyPI, where deterministic, offline tests are required to certify software correctness in sandboxed environments. In contrast, `wbopendata` operates in trusted, interactive Stata environments where network access is both available and essential. Because the core functionality of the command is to retrieve and process live data, operational validation against the actual API is the most informative way to assess reliability.

Under this paradigm, test failures do not necessarily imply software defects. They may reflect upstream API outages, schema changes, or metadata revisions. The objective of the test suite is therefore not to freeze outputs, but to make such changes explicit and diagnosable.

C.2 Dual testing paradigm: live and deterministic

The test suite employs two complementary testing paradigms. The live integration tests described above detect upstream regressions in real time but are inherently non-deterministic: results depend on current API responses, network conditions, and the evolving state of upstream data. To complement this approach, version 18.1 introduces *deterministic offline tests* (category DET) that execute against pre-recorded CSV fixtures stored in `qa/fixtures/`, eliminating network dependency entirely.

Each fixture captures a specific API response state—for example, a known dataset for a particular indicator-country-year combination, or an empty response simulating a deprecated indicator. The `offline()` option directs the command to load data from the fixture directory rather than querying the live API. Tests using fixtures produce identical results regardless of network conditions or upstream data revisions, enabling exact value pinning and regression baselines that would be impossible with live tests.

For error condition tests (category ERR), the suite employs Stata's `rcof` certification command, an undocumented testing primitive consistent with the methodology described by `?`. Unlike `capture noisily`, which merely records whether a command succeeded or failed, `rcof` verifies the *exact* return code produced:

```
. rcof "noi wbopendata, clear" == 198
```

This confirms that omitting a required option produces a syntax error (rc 198) rather than a network timeout or other incidental failure. The distinction matters: the old `capture noisily` pattern would pass even if the command errored for the wrong reason, while `rcof` guarantees that the error originates from the intended validation logic.

Because `rcof` halts execution on mismatch, integration with the test harness's "continue on failure" convention requires wrapping it in `capture noisily`:

```
. capture noisily {
.     rcof "noi wbopendata, clear" == 198
. }
. if _rc == 0 test_pass
. else test_fail "Expected rc 198 (syntax error)"
```

This pattern preserves `rcof`'s precision while allowing the test suite to continue executing subsequent tests after a failure.

The combination of live and deterministic tests creates a dual-paradigm framework: live tests surface upstream changes that require attention, while deterministic tests provide stable, reproducible baselines. Together with the extreme-case tests (category EXT), which exercise boundary conditions such as minimal queries, large result sets, and edge inputs, these additions bring the total suite to 92 tests across 15 categories.

C.3 Test harness structure

The primary QA suite is implemented in `qa/run_tests.do`, which supports full test runs, single-test execution, and verbose (trace) modes. Tests are organized into functional categories that correspond to core features of the command, including environment checks, download modes, output formatting, country metadata, graph metadata, update commands, language options, and advanced features.

Each test follows a standardized pattern:

- explicit execution of the command under test,
- immediate inspection of return codes,
- verification of expected variables, structures, or scalars,
- informative pass/fail messages tied to specific failure modes.

Helper routines manage test execution, result aggregation, and logging, ensuring that failures are attributed to specific test identifiers. A complete test run produces a timestamped log and updates a cumulative test history file.

Test category definitions

Table **??** organizes the 163 tests into 15 functional categories, ordered by a typical analytical workflow: environment setup, data discovery, retrieval, post-processing, infrastructure, and validation. Each category is defined below.

- **ENV** — *Environment.* Validates prerequisites: network connectivity, API reachability, and presence of local YAML metadata files.
- **DISC** — *Discovery.* Exercises the indicator search and metadata lookup subsystem: `sources`, `alltopics`, `search()`, `info()`, and their filtering options (`searchsource`, `searchtopic`, `searchfield`, `exact`, `detail`).
- **DL** — *Download modes.* Tests core data retrieval paths: single-indicator, multi-indicator, country-specific, and topic-based downloads in both wide and long layouts.
- **FMT** — *Output format.* Verifies output structure options including `long`, `year()`, and `latest`.
- **CTRY** — *Country options.* Tests metadata enrichment via `match()`, `full`, `geo`, `capital`, ISO code handling, and regional/income classification variables.
- **TOPIC/LANG** — *Topics & language.* Covers topic-filtered downloads and non-English metadata retrieval (`language(es)`).

- **LW** — *Linewrap.* Validates the `linewrap()` graph annotation system: metadata field wrapping, `maxlength()`, multi-indicator stacking, and `latest` scalar returns.
- **CHAR** — *Characteristics.* Verifies that Stata `char` metadata is attached to datasets and variables by default, persists across operations, and is suppressed by `nochar`.
- **Advanced** — *Advanced features.* Tests specialized options: `projection`, `nobasic`, `nometadata`, `describe`, and date handling.
- **CACHE/SYNC** — *Cache & sync.* Exercises the full cache lifecycle (`nocache`, `cachedays()`, `cleardatacache`, `resetdatacache`), metadata synchronization (`sync`, `checkupdate`), and cross-validation between cache layers.
- **UPD** — *Maintenance.* Tests administrative commands including legacy `update` aliases, `describe`, and sync interoperability.
- **REG** — *Regression.* Permanent tests for historically resolved bugs (issues #33, #45, #46, #51), ensuring fixes are never lost.
- **ERR** — *Error conditions.* Validates that invalid inputs produce correct return codes using Stata's `rcof` certification command.
- **EXT** — *Extreme cases.* Exercises boundary conditions: minimal queries, large result sets, and unusual input combinations.
- **DET** — *Deterministic.* Offline tests using pre-recorded CSV fixtures, enabling exact value pinning independent of network state.

C.4 Regression testing and cumulative reliability

A key feature of the QA framework is the systematic treatment of historical bugs as permanent regression tests, following **?**'s principle that "tests are never thrown away". Issues such as failures in multi-indicator queries, metadata parsing errors, incorrect handling of deprecated indicators, or option incompatibilities are encoded as named test cases (e.g., `REG-33`, `REG-45`, `REG-46`, `REG-51`).

This approach ensures that fixes are not transient. Once a bug is resolved, the corresponding test remains in the suite and must pass in all subsequent runs. As a result, reliability is cumulative rather than episodic.

The longitudinal test history recorded in `test_history.txt` documents this process explicitly: early versions show multiple failures as new features were introduced and edge cases identified, followed by successive reductions in failures as fixes were applied, culminating in fully passing test runs for recent releases. This history provides an auditable record of stabilization over time rather than a single-point snapshot.

C.5 Diagnostics and failure modes

Tests are designed to capture and report informative diagnostics rather than binary outcomes. When a test fails, logs record the API endpoint accessed, the parameters supplied, and the return codes or error messages produced. This information is essential for distinguishing between upstream changes and genuine software regressions.

Illustrative examples of user-facing diagnostics—including missing indicators, deprecated series, and metadata update behavior—are reported in Appendix **??**. Appendix **??** documents *what users see*, while the present appendix documents *how those behaviors are systematically validated.*

C.6 Indicator lifecycle and metadata evolution

The QA framework explicitly accounts for the fact that the set of available indicator codes and their associated metadata evolve over time. Indicators may be deprecated, archived, or reclassified as source databases are revised, and metadata fields may be corrected or expanded.

Tests covering maintenance commands (`sync`, `checkupdate`, `sync replace`, and the legacy `update` aliases) verify that such changes are detected and propagated through the local cache. Deprecated indicators are expected to trigger informative diagnostics rather than silent failures. By treating indicator lifecycle events as testable behavior, the framework reduces the risk that analytical pipelines continue to rely unknowingly on obsolete series or outdated definitions.

C.7 Reproducibility of the QA process

All QA artifacts—including test scripts, protocols, logs, and cumulative test histories—are version-controlled alongside the source code. This allows the QA process itself to be reproduced, inspected, and audited.

Because tests depend on live API responses, exact outputs may change as upstream data are revised. Such changes are treated as expected signals when they reflect documented updates. The purpose of the QA framework is therefore not to enforce static results, but to ensure that change is explicit, traceable, and consistent with upstream revisions.

C.8 Continuous validation under ecosystem constraints

Although `wbopendata` is developed within the Stata ecosystem, which lacks the automated release-gate infrastructure common to CRAN or PyPI, its QA framework implements the functional equivalent of continuous integration and continuous validation—consistent with **?**'s demonstration that rigorous software certification is achievable through disciplined testing practices rather than formal automation. Automated test execution, regression protection, and versioned test histories ensure that each code change is eval-

uated against both current functionality and historically fixed issues.

In this setting, continuous validation is achieved not through centralized CI/CD services, but through reproducible test scripts that can be executed consistently by developers prior to release. Each test run produces a timestamped log and updates a cumulative history, creating an auditable trail of stability across versions. This approach aligns with institutional constraints—such as the need for live API access and trusted execution environments—while preserving the core objective of CI/CD practices: early detection of regressions and disciplined release management.

By adapting CI/CD principles to the realities of statistical software distribution, `wbopendata` demonstrates that rigorous quality assurance is achievable even in environments without formal automation pipelines.

C.9 Illustrative QA artifacts

To support transparency, selected excerpts from the automated test harness and logs are included below. These excerpts illustrate the structure of the testing framework and the form of diagnostics produced during execution.

Test harness initialization (excerpt)

```
------------------------------------------------------------------------------
WBOPENDATA TEST SUITE
Version: 18.1.0
Build date: 10Feb2026
Date:  9 Feb 2026 14:41:01
Stata: 17
------------------------------------------------------------------------------

. run_test "ENV-01" "wbopendata version matches repo"

--- TEST ENV-01: wbopendata version matches repo ---
Installed version: *! v 18.1.0  10Feb2026
Repo version:      *! v 18.1.0  10Feb2026
PASS
r; t=0.00 14:41:01

------------------------------------------------------------------------------

. run_test "DL-01" "Single indicator download"

--- TEST DL-01: Single indicator download ---
. wbopendata, indicator(SP.POP.TOTL) clear nometadata long
Observations: 17,290
Variables:    12
PASS
r; t=4.27 14:41:05

------------------------------------------------------------------------------

. run_test "DL-04" "Multiple indicators download"
```

```
--- TEST DL-04: Multiple indicators download ---
. wbopendata, indicator(SP.POP.TOTL;NY.GDP.MKTP.CD) clear long nometadata
PASS
r; t=5.92 14:41:14

--------------------------------------------------------------------------------

. run_test "DISC-01" "Search basic keyword"

--- TEST DISC-01: Search basic keyword ---
. _wbopendata_search GDP, limit(5)
Found 677 results, displayed 5, first=6.0.GDP_current
PASS
r; t=0.26 14:44:44

--------------------------------------------------------------------------------

. run_test "SYNC-02" "Sync replace command"

--- TEST SYNC-02: Sync replace command ---
. wbopendata, sync replace
sync rc: 0
Cache created after sync: yes
PASS
r; t=4.67 14:44:21

--------------------------------------------------------------------------------

. run_test "UPD-01" "Update query command"

--- TEST UPD-01: Update query command ---
. wbopendata, update query
Indicators update status: 29323 indicators, no updates available
PASS
r; t=0.07 14:41:52

--------------------------------------------------------------------------------

TEST SUMMARY

Tests Run:    89
Tests Passed: 89
Tests Failed: 0

ALL TESTS PASSED!

--------------------------------------------------------------------------------
```

Summary of a complete test run (excerpt)

```
--------------------------------------------------------------------
. wbopendata Automated Test Suite - Summary Report
--------------------------------------------------------------------
  ENV:   Environment checks ................. 5/5 passed
  DL:    Download modes ..................... 5/5 passed
  FMT:   Output format ...................... 3/3 passed
  CTRY:  Country attributes ................. 10/10 passed
  REG:   Regression fixes ................... 4/4 passed
```

```
  LW:    Linewrap metadata .................. 4/4 passed
  UPD:   Maintenance commands ............... 6/6 passed
  TOPIC: Topics and language ................ 2/2 passed
  ADV:   Advanced features .................. 6/6 passed
  CACHE: Cache system (v18.0) ............... 8/8 passed
  SYNC:  Sync system (v18.0) ................ 5/5 passed
  DISC:  Discovery commands (v18.0) ......... 7/7 passed
  CHAR:  Characteristic metadata (v18.1) .... 6/6 passed
  ERR:   Error conditions (v18.1) ........... 8/8 passed
  EXT:   Extreme cases (v18.1) .............. 4/4 passed
  DET:   Deterministic/offline (v18.1) ...... 6/6 passed
------------------------------------------------------------------------
Total:    89 tests executed
Passed:   89 tests (100%)
Failed:   0 tests
Duration: 4 minutes 12 seconds
------------------------------------------------------------------------
Result: ALL TESTS PASSED
------------------------------------------------------------------------
```

C.10 Help-file example validation

A complementary test suite, `tests/test_help_examples.do`, validates that every example documented in the help file (`wbopendata.sthlp`) executes without error. This suite comprises 71 tests organized in 12 categories:

- **Categories 1–3**: Discovery commands, deprecated options, and cache management (local YAML lookups, no API calls required).
- **Categories 4–6**: Live data downloads testing indicator, country, topic, and multi-indicator queries with various option combinations.
- **Categories 7–8**: Named click-to-run examples and the end-to-end discovery workflow (sources → search → info → download).
- **Categories 9–12**: Inline example replay with data assertions, stored results verification, characteristic metadata checks, and data response cache workflows.

The script includes a version guard that compares the test version against the resolved `wbopendata.ado` version and exits on mismatch, preventing stale tests from running against newer code. Two execution modes are supported: *dev* mode (default) manipulates the `adopath` to prioritize development source files; *installed* mode tests the `net install`-ed copy from PLUS. This dual-mode design ensures that both development and release configurations are validated.

The help-file suite serves a distinct purpose from the QA suite: while `run_tests.do` validates *features*, `test_help_examples.do` validates *documentation*. If a code change breaks a help-file example, this suite detects the divergence before users encounter it.

C.11 Implications for reproducible analytics

The QA practices documented here reinforce the paper's central argument that reproducibility depends not only on statistical methods, but also on the reliability of data-access infrastructure. By embedding operational validation, regression testing, and explicit diagnostics into the development lifecycle, `wbopendata` ensures that scripted data acquisition remains a stable and auditable foundation for empirical analysis, even as upstream data systems evolve.

In this sense, quality assurance is not an ancillary engineering concern, but a core component of reproducible analytics. It provides the enforcement mechanism through which principles such as data acquisition as code and explicit data provenance are sustained in practice.